



🍷 Set Object Poses

`SetObjectPoses` is useful for setting a scene to the exact state of a previous initialization without relying on the Object Position Randomization's random seed. Use a previous metadata dump of SimObjects to get position/rotation data for each SimObject. Sets up a scene according to the provided objects and poses. Objects which are specified twice will be copied.

Warning

All moveable and pickupable objects will be removed from the scene if they are not specified.

```
controller.step(  
    action='SetObjectPoses',  
    objectPoses=[  
        {  
            "objectName": "Alarm_Clock_19",  
            "rotation": {  
                "y": 270,  
                "x": 0,  
                "z": 0  
            },  
            "position": {  
                "y": 0.8197357,  
                "x": 2.45610785,  
                "z": 0.054755792  
            }  
        },  
        {  
            "objectName": "Alarm_Clock_19",  
            "rotation": {  
                "y": 270,  
                "x": 0,  
                "z": 0  
            },  
            "position": {  
                "y": 0.8197357,  
                "x": 2.64169645,  
                "z": -0.134690791  
            }  
        }  
    ],  
    {...},  
    {  
        "objectName": "Alarm_Clock_19",  
        "rotation": {  
            "y": 270,  
            "x": 0,  
            "z": 0  
        },  
        "position": {  
            "y": 0.8197357,  
            "x": 2.64169645,  
            "z": -0.134690791  
        }  
    }  
])  
)
```

🧸 Set Mass Properties

`SetMassProperties` changes the mass properties of any `Pickupable` or `Moveable` object.

```
controller.step(  
    action="SetMassProperties",  
    objectId="Apple|+1.25|+0.25|-0.75",  
    mass=22.5,  
    drag=15.9,  
    angularDrag=5.6  
)
```

Set Object Poses Parameters

- objectPoses:** list[dict[str, any]] required
- List of object names and their poses. This information can be dumped from the metadata of a prior episode. Each pose must contain keys for
- objectName:** The name of the Sim Object reported by the object metadata's `name` parameter.
 - position:** Global coordinate position of the object, as reported in the object's `position` metadata.
 - rotation:** Local coordinate rotation of the object, as reported in the object's `rotation` metadata.

🔧 Set Temperature

🔍 Object Type Temperature Decay Time

`SetRoomTempDecayTimeForType` changes all objects specified by objectType in the current scene to have a new Room Temp Decay Time specified by TimeUntilRoomTemp. This can be done to adjust the amount of time it takes for specific object types to return to room temperature. By default, all objects begin at Room Temperature. Objects placed on/in Hot or Cold sources (stove burner, microwave, fridge etc.) will have their ObjectTemperature value changed to Hot or Cold. If the object is removed from a Hot or Cold source, they will return to room temperature over time. The default time it takes for an object to decay to Room Temp is 10 seconds.

```
controller.step(  
    action="SetRoomTempDecayTimeForType",  
    objectType="Bread",  
    TimeUntilRoomTemp=20.0  
)
```

Parameters

- objectType:** str
- The object type to change the decay timer of. See a full list of Object Types on the Object Types page.
- TimeUntilRoomTemp:** float required
- The amount of time it will take for an object to decay from Hot/Cold to Room Temp.

🌐 Global Temperature Decay Time

`SetGlobalRoomTempDecayTime` changes all objects in the current scene to have a new Room Temp Decay Time specified by TimeUntilRoomTemp. By default, all objects begin at Room Temperature. Objects placed on/in Hot or Cold sources (stove burner, microwave, fridge etc.) will have their ObjectTemperature value changed to Hot or Cold. If the object is removed from a Hot or Cold source, they will return to room temperature over time. The default time it takes for an object to decay to Room Temp is 10 seconds.

```
controller.step(  
    action="SetGlobalRoomTempDecayTime",  
    TimeUntilRoomTemp=20.0  
)
```

Parameters

- TimeUntilRoomTemp:** float required
- The amount of time it will take for an object to decay from Hot/Cold to Room Temp.

🍳 Disable Temperature Decay

`SetDecayTemperatureBool` disables the decay over time of the ObjectTemperature of objects. If set to False, objects will not decay to room temperature and will remain Hot/Cold even if removed from the Hot/Cold source.

```
controller.step(  
    action="SetDecayTemperatureBool",  
    allowDecayTemperature=False  
)
```

- allowDecayTemperature:** bool = True
- Set to allow object Temperatures to decay over time.

🤖 Hiding Objects

Warning

Hiding an object may cause unintended interactions with other objects after an object is disabled. For example, a table with a Plate and Apple resting on top of it will cause both the Plate and Apple to fall to the floor if the table object supporting them is disabled.

❌ Remove

`RemoveFromScene` completely destroys an removes an object from the scene. This action cannot be undone (without resetting the scene).

```
controller.step(  
    action="RemoveFromScene",  
    objectId="Mug|+0.25|-0.27|+1.05"  
)
```

Remove from Scene Parameters

- objectId:** str required
- The unique identifier of the object in the scene.

🔒 Disable

`DisableObject` disables an object from being visible in the scene. Unlike the RemoveFromScene action, this does not permanently destroy an object in the scene, as it can be toggled back on later in the episode using EnableObject.

```
controller.step(  
    action="DisableObject",  
    objectId="DiningTable|+1.0|+1.0|+1.0"  
)
```

Disable Objet Parameters

- objectId:** str required
- The string id of the sim object to disable. Note that this may cause unintended interactions with other objects after an object is disabled. For example, a table with a Plate and Apple resting on top of it will cause both the Plate and Apple to fall to the floor if the table object supporting them is disabled.

😄 Enable

`EnableObject` activates an object if it has been previously disabled by DisableObject.

```
controller.step(  
    action="EnableObject",  
    objectId="DiningTable|+1.0|+1.0|+1.0"  
)
```

Enable Object Parameters

- objectId:** str required
- The string id of the sim object to reactivate. Note that this may cause unintended interactions with other objects after an object is reactivated. Enabled objects will return to their original location, which may cause clipping or weird collision with other objects if another sim object or the agent has moved into the area where the enabled object is.